

Name: Kaustubh Arvind Kadam

Roll no.: S20240010114

Section: 2

CCN Assignment

CODE

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define INF 9999
```

```
typedef struct Node {
```

```
    int vertex, dist;
```

```
    struct Node* next;
```

```
} Node;
```

```
Node* createNode(int v, int d) {
```

```
    Node* n = malloc(sizeof(Node));
```

```
    n->vertex = v; n->dist = d; n->next = NULL;
```

```
    return n;
```

```
}
```

```
void pq_insert(Node** head, int v, int d) {
```

```
    Node* n = createNode(v, d);
```

```
    n->next = *head;
```

```
    *head = n;
```

```
}
```

```
/* returns vertex with minimum dist and removes it */
```

```
int pq_extractMin(Node** head) {
```

```
    Node *curr = *head, *prev = NULL;
```

```
    Node *minNode = *head, *minPrev = NULL;
```

```
    while (curr) {
```

```
        if (curr->dist < minNode->dist) {
```

```
            minNode = curr; minPrev = prev;
```

```
        }
```

```
        prev = curr; curr = curr->next;
```

```
    }
```

```
    int v = minNode->vertex;
```

```
    if (minPrev == NULL) *head = minNode->next;
```

```
    else minPrev->next = minNode->next;
```

```
    free(minNode);
```

```
    return v;
```

```
}
```

```
void pq_updateDist(Node* head, int v, int d) {
```

```
    while (head) {
```

```
        if (head->vertex == v) { head->dist = d; return; }
```

```
        head = head->next;
```

```
    }
```

```
}
```

```
void printPath(int parent[], int j) {
```

```
    if (parent[j] == j) { printf("%d", j); return; }
```

```
    printPath(parent, parent[j]);
```

```
    printf(" -> %d", j);
```

```
}
```

```
void dijkstra(int** graph, int V, int src, int dest) {  
    int dist[V], parent[V], visited[V];  
    Node* pq = NULL;  
  
    for (int i = 0; i < V; i++) {  
        dist[i] = INF; visited[i] = 0;  
        parent[i] = i;    /* every node points to itself initially */  
        pq_insert(&pq, i, INF);  
    }  
    dist[src] = 0;  
    pq_updateDist(pq, src, 0);  
  
    while (pq) {  
        int u = pq_extractMin(&pq);  
        if (dist[u] == INF) break; /* remaining nodes unreachable */  
        visited[u] = 1;  
  
        for (int v = 0; v < V; v++) {  
            if (!visited[v] && graph[u][v] != INF) {  
                int nd = dist[u] + graph[u][v];  
                if (nd < dist[v]) {  
                    dist[v] = nd;  
                    parent[v] = u;  
                    pq_updateDist(pq, v, nd);  
                }  
            }  
        }  
    }  
}
```

```

if (dist[dest] == INF) {
    printf("\nNo path from %d to %d\n", src, dest);
    return;
}

printf("\nShortest distance (Dijkstra): %d\n", dist[dest]);
printf("Path: "); printPath(parent, dest); printf("\n");
}

```

```

void bellmanFord(int** graph, int V, int src, int dest) {
    int dist[V], parent[V];

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        parent[i] = i;    /* self-loop sentinel */
    }
    dist[src] = 0;

    /* relax V-1 times */
    for (int k = 0; k < V - 1; k++) {
        int updated = 0;
        for (int i = 0; i < V; i++) {
            if (dist[i] == INF) continue; /* skip unreachable */
            for (int j = 0; j < V; j++) {
                if (i == j) continue; /* skip self-loops */
                if (graph[i][j] == INF) continue;
                if (dist[i] + graph[i][j] < dist[j]) {
                    dist[j] = dist[i] + graph[i][j];
                    parent[j] = i;
                    updated = 1;
                }
            }
        }
    }
}

```

```

        }
    }
}

if (!updated) break; /* early exit if stable */
}

/* negative-cycle check */
for (int i = 0; i < V; i++) {
    if (dist[i] == INF) continue;
    for (int j = 0; j < V; j++) {
        if (i == j) continue;
        if (graph[i][j] == INF) continue;
        if (dist[i] + graph[i][j] < dist[j]) {
            printf("\nShortest path is undefined.\n");

            return;
        }
    }
}

if (dist[dest] == INF) {
    printf("\nNo path from %d to %d\n", src, dest);
    return;
}

printf("\nShortest distance (Bellman-Ford): %d\n", dist[dest]);
printf("Path: "); printPath(parent, dest); printf("\n");
}

int hasNegative(int** graph, int V) {

```

```

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            if (graph[i][j] < 0 && graph[i][j] != INF) return 1;
    return 0;
}

```

```

void printGraph(int** graph, int V) {
    printf("\nAdjacency Matrix:\n  ");
    for (int i = 0; i < V; i++) printf("%5d ", i);
    printf("\n");
    for (int i = 0; i < V; i++) {
        printf("[%2d] ", i);
        for (int j = 0; j < V; j++) printf("%5d ", graph[i][j]);
        printf("\n");
    }
}

```

```

int** makeGraph(int V) {
    int** g = malloc(V * sizeof(int*));
    for (int i = 0; i < V; i++) g[i] = malloc(V * sizeof(int));
    return g;
}

```

```

int** addNode(int** graph, int* V) {
    int newV = *V + 1;
    int** ng = makeGraph(newV);

    for (int i = 0; i < newV; i++)
        for (int j = 0; j < newV; j++)
            ng[i][j] = (i < *V && j < *V) ? graph[i][j] : INF;
    ng[*V][*V] = 0;
}

```

```

printf("\nEnter weights for new node %d (9999 = no edge):\n", *V);
for (int i = 0; i < *V; i++) {
    int w;

    printf(" Weight between %d and %d: ", i, *V);
    scanf("%d", &w);
    ng[i][*V] = w;
    ng[*V][i] = w;
}

/* free old graph */
for (int i = 0; i < *V; i++) free(graph[i]);
free(graph);

*V = newV;
printf("Node %d added.\n", *V - 1);
return ng;
}

int** deleteNode(int** graph, int* V) {
    int del;

    printf("Enter node to delete (0-%d): ", *V - 1);
    scanf("%d", &del);

    if (del < 0 || del >= *V) {
        printf("Invalid node.\n");
        return graph;
    }

    int newV = *V - 1;
    int** ng = makeGraph(newV);

```

```

int r = 0;
for (int i = 0; i < *V; i++) {
    if (i == del) continue;
    int c = 0;
    for (int j = 0; j < *V; j++) {
        if (j == del) continue;
        ng[r][c++] = graph[i][j];
    }
    r++;
}

for (int i = 0; i < *V; i++) free(graph[i]);
free(graph);

*V = newV;
printf("Node %d deleted.\n", del);
return ng;
}

int main() {
    int V;
    printf("Enter number of nodes: ");
    scanf("%d", &V);

    int** graph = makeGraph(V);

    printf("Enter adjacency matrix (9999 = no edge, 0 on diagonal):\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)

```



```

scanf("%d", &graph[i][j]);

int choice;
while (1) {
    printf("\n1. Shortest Path\n2. Add Node\n"
           "3. Delete Node\n4. Print Graph\n5. Exit\n");
    printf("Choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        int src, dest;
        printf("Enter source and destination: ");
        scanf("%d %d", &src, &dest);

        if (src < 0 || src >= V || dest < 0 || dest >= V) {
            printf("Invalid node(s). Range: 0-%d\n", V - 1);
            continue;
        }
        if (src == dest) {
            printf("Source and destination are the same (cost 0).\n");
            continue;
        }

        if (hasNegative(graph, V))
            bellmanFord(graph, V, src, dest);
        else
            dijkstra(graph, V, src, dest);
    }
    else if (choice == 2) graph = addNode(graph, &V);
    else if (choice == 3) graph = deleteNode(graph, &V);
    else if (choice == 4) printGraph(graph, V);
}

```

```

        else { printf("Exiting.\n"); break; }
    }

    for (int i = 0; i < V; i++) free(graph[i]);
    free(graph);
    return 0;
}

```

Output Screenshots

```

● kaustubh@Kaustubh:~/academics/CCN_assignment$ ./a.out
Enter number of nodes: 4
Enter adjacency matrix (9999 = no edge, 0 on diagonal):
0 2 9999 1
0 2 3 2
9999 3 0 4
1 2 4 0

1. Shortest Path
2. Add Node
3. Delete Node
4. Print Graph
5. Exit
Choice: 1
Enter source and destination: 0 2

Shortest distance (Dijkstra): 5
Path: 0 -> 3 -> 2

1. Shortest Path
2. Add Node
3. Delete Node
4. Print Graph
5. Exit
Choice: 2

Enter weights for new node 4 (9999 = no edge):
Weight between 0 and 4: 3
Weight between 1 and 4: 9999
Weight between 2 and 4: 2
Weight between 3 and 4: 1
Node 4 added.

```

```
1. Shortest Path
2. Add Node
3. Delete Node
4. Print Graph
5. Exit
```

Choice: 1

Enter source and destination: 0 2

Shortest distance (Dijkstra): 4

Path: 0 -> 3 -> 4 -> 2

```
1. Shortest Path
2. Add Node
3. Delete Node
4. Print Graph
5. Exit
```

Choice: 3

Enter node to delete (0-4): 4

Node 4 deleted.

```
1. Shortest Path
2. Add Node
3. Delete Node
4. Print Graph
5. Exit
```

Choice: 1

Enter source and destination: 0 2

Shortest distance (Dijkstra): 5

Path: 0 -> 3 -> 2

```
● kaustubh@Kaustubh:~/academics/CCN_assignment$ ./a.out
```

Enter number of nodes: 3

Enter adjacency matrix (9999 = no edge, 0 on diagonal):

0 4 9999

4 0 -2

9999 -2 0

```
1. Shortest Path
2. Add Node
3. Delete Node
4. Print Graph
5. Exit
```

Choice: 1

Enter source and destination: 0 2

Shortest path is undefined.

```
1. Shortest Path
2. Add Node
3. Delete Node
4. Print Graph
5. Exit
```

Choice: 5

Exiting.

Q2 and Q3

Q2/ Datagram are limited to 1500 bytes, which include the header.

The absolute maximum size of a single packet traversing the network from Host A to Host B can't exceed 1500 bytes.

Total datagram limit = 1500 bytes

IP header overhead = 20 bytes

Available payload (MP3) = 1480 bytes

$$\text{No. of Datagrams} = \frac{5000000}{1480} = 3378.378$$

1) 3378 full datagrams $\rightarrow$ carrying 4999440 bytes of MP3 data (3378×1480)

2) This leaves a remaining 560 bytes of MP3 need to be transmitted.

So rounding of 3378.378 to 3379

$\therefore$ Datagram 1 through 3,378

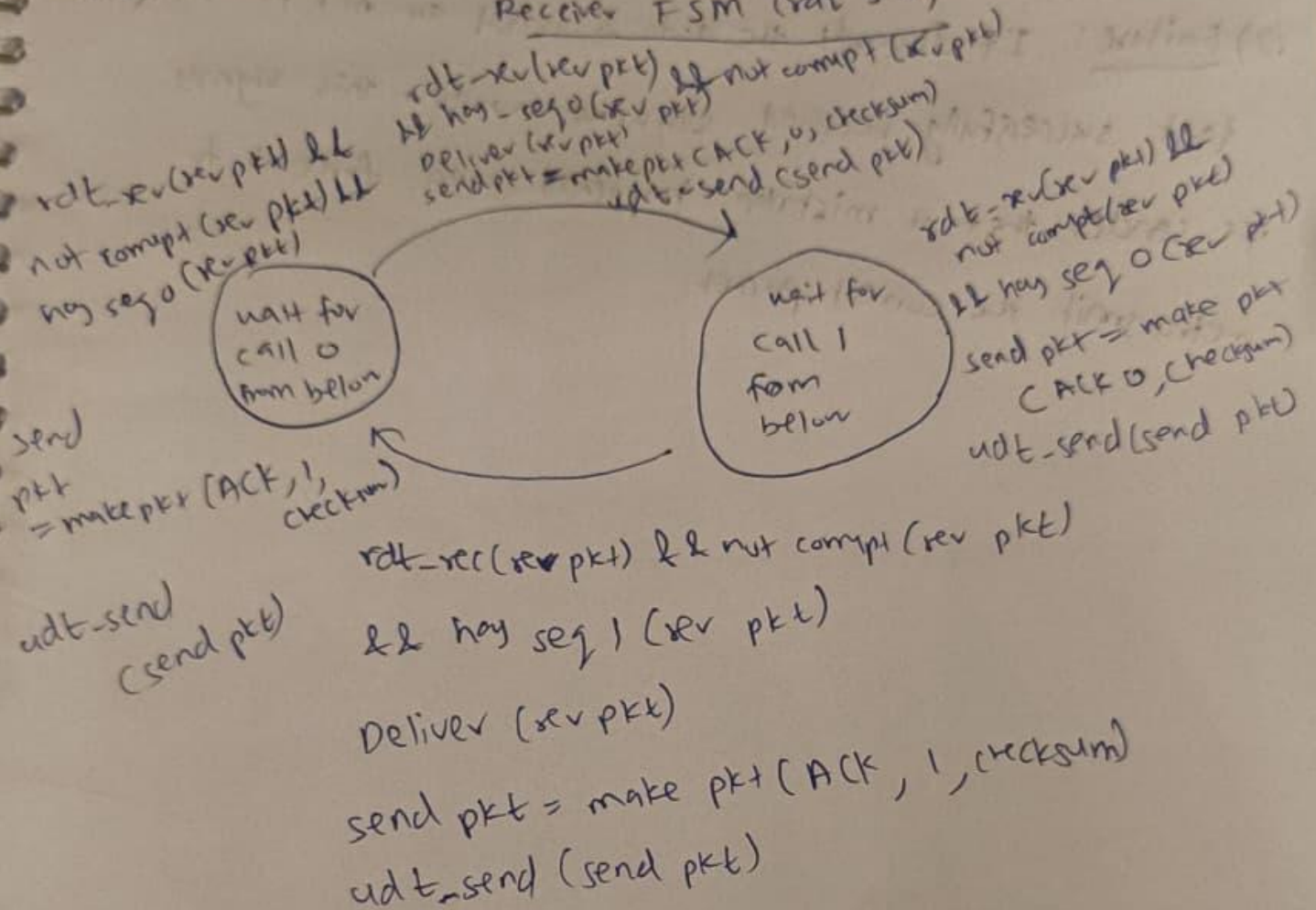
$\therefore$ (20 bytes header + 1480 bytes of MP3 data)

Datagram, 3379 (final packet)

∴ (20 bytes header + 560 bytes of remaining mp3 data)

Q3)

Receiver FSM (rdt 3.0)



● RDT 3.0 is an alternating bit protocol. The receiver is simple & constant toggles between 2 states, waiting for 0 and waiting for 1.

Depending on what arrives, it takes one of 2 actions:

1) Success: It extracts the data, delivers it to the app, sends an ACK for that specific sequence number and flip its state to wait for the opposite number.

2) Failure: It discards the data & sends an ACK for the last successfully received sequence number. This signals the sender that a mistake happened and forces it to retransmit the correct packet.

